

CS543 · TUTORIAL 2 OF 28

Setting Up, and Sending HTTP by Hand

Install everything, verify it works, then read raw requests and responses

1 hour | Tutorial for Lecture 1 · Foundations

Bring a laptop. Every command in this deck is meant to be typed.

Dr. Pramit Mazumdar · Dept. of CSE · IIIT Vadodara

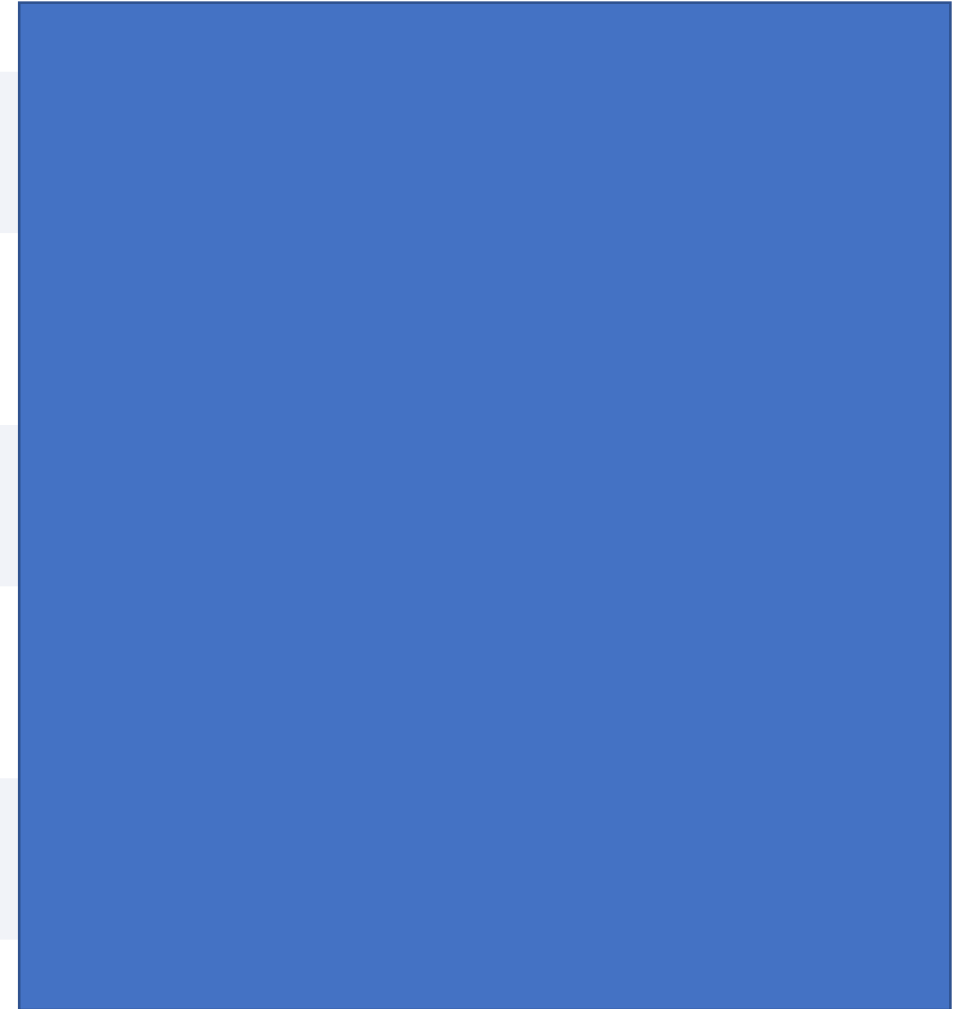
1 What are the three parts of an HTTP request line?

2 Which side of a conversation always speaks first?

3 What does "HTTP is stateless" mean in practice?

4 Which header tells the server how to parse your body?

5 Does `fetch` throw an error on a 404?



What you need installed, and why

Tool	What it is for	How to get it	Check it worked
Node.js (LTS)	Runs JavaScript outside the browser. Every server you build needs it.	nodejs.org → download the LTS installer → accept all defaults → RESTART your terminal	<code>node --version</code> <code>v22.x.x</code> (any v20+)
npm	Installs libraries. Comes bundled with Node — nothing separate to install.	Installed automatically with Node.js above. Do not install it separately.	<code>npm --version</code> <code>10.x.x</code>
curl	Sends raw HTTP from the terminal. Shows the literal bytes on the wire.	Already present on Windows 10+, macOS and Linux. Nothing to install for most of you.	<code>curl --version</code> <code>curl 8.x.x ...</code>
VS Code	Editor. We also use its REST Client extension as a friendlier API tool.	code.visualstudio.com → install → Extensions panel → search "REST Client" (by Huachao Mao) → Install	<code>Ctrl+Shift+X</code> shows the extension as Installed

Where to type these commands?

WINDOWS

- Use PowerShell, not Command Prompt
- Open it: press Start, type "powershell", press Enter
- Or in VS Code: Terminal → New Terminal
- WSL (Ubuntu) also works if you have it

```
Start → powershell → Enter
```

macOS

- Use Terminal, or iTerm2 if you have it
- Open it: Cmd+Space, type "terminal", press Enter
- Or in VS Code: Terminal → New Terminal
- The default shell is zsh — that is fine

```
Cmd+Space → terminal
```

LINUX

- Any terminal emulator
- Usually Ctrl+Alt+T opens one
- Or in VS Code: Terminal → New Terminal
- bash or zsh, either is fine

```
Ctrl+Alt+T
```

TERMINAL NAVIGATION — the six commands you need

```
pwd
```

print working directory — where am I right now?

```
cd ..
```

go up one level

```
ls (dir on Windows CMD)
```

list the files in this directory

```
mkdir foldername
```

create a new folder here

```
cd foldername
```

go into that folder

```
clear (cls on Windows)
```

clear the screen

Verify the setups

1 Open a terminal

```
(see the previous slide  
for your machine)
```

A prompt appears, ending in \$ or >

2 Check Node is installed

```
node --version
```

v22.14.0 — any v20 or higher is fine

3 Check npm came with it

```
npm --version
```

10.9.2 — any 9 or higher is fine

4 Check curl works (Windows: curl.exe)

```
curl --version
```

curl 8.7.1 (x86_64...) plus a list of protocols

5 Make a working folder

```
mkdir cs543  
cd cs543
```

No output at all. That means it worked.

6 Confirm where you are

```
pwd
```

A path ending in /cs543 or \cs543

Demonstration

The command, and everything it prints back

```
$ curl -i
https://jsonplaceholder.typicode.com/users/1

HTTP/2 200                                <- status line
content-type: application/json; charset=utf-8
cache-control: max-age=43200
content-length: 509                        <- headers
                                           <- blank line
{                                           <- body starts here
  "id": 1,
  "name": "Leanne Graham",
  "email": "Sincere@april.biz"
}
```

Why curl before Postman

Postman is more comfortable and you will use it all semester. But it hides the message.

curl shows the literal bytes going over the wire, which is what you must reason about when something does not work.

THE SIX FLAGS YOU NEED ALL SEMESTER

- i** include the response headers in the output
`curl.exe -i URL`
- v** verbose: show the REQUEST you sent as well
`curl.exe -v URL`
- X** set the method (default is GET)
`curl.exe -X POST URL`
- H** add one header; repeat for more
`curl.exe -H "Accept: application/json" URL`
- d** send a request body (implies POST)
`curl.exe -d '{"a":1}' URL`
- I** capital i: HEAD request, headers only
`curl.exe -I URL`

Sending a body — all on one line

```
curl.exe -i -X POST -H "Content-Type:
application/json" \
  -d '{"name":"Asha"}' \
  https://jsonplaceholder.typicode.com/users

# Windows PowerShell: use " not ' around the JSON,
# and escape inner quotes: -d '{"name":"Asha"}'
```

VS Code REST Client

1

Open VS Code in your folder

```
code .  
(or File → Open Folder)
```

VS Code opens showing the cs543 folder

2

Install the extension

```
Ctrl+Shift+X, search  
"REST Client"
```

Publisher is Huachao Mao. Click Install.

3

Create a request file

```
New file, save it as  
requests.http
```

The name must end in .http

4

Type a request into it

```
GET https://jsonplaceholder  
.typicode.com/users/1
```

A small "Send Request" link appears above the line

5

Click Send Request

```
(click the link, or  
Ctrl+Alt+R)
```

The full response opens in a side panel

6

Add a POST below it

```
Separate requests with  
a line of ###
```

Each request gets its own Send Request link

Problem set A — read the message

- 1 Run the verbose form and list every header YOUR machine sent. Which did you not ask for?

```
curl.exe -v  
https://jsonplaceholder.typicode.com/posts/1
```

- 2 Request a post that does not exist. What status comes back, and what is in the body?

```
curl.exe -i  
https://jsonplaceholder.typicode.com/posts/9999
```

- 3 POST a JSON body WITHOUT a Content-Type header, then WITH one. Compare the replies.

```
curl.exe -i -X POST -d '{"title":"hi"}' <URL>/posts
```

- 4 Find a response that includes a Location header. Which status accompanies it?

```
look at what your POST in step 3 returned
```

- 5 Fetch only the headers of a large page. Why would a program want headers and no body?

```
curl.exe -I https://www.iiitvadodara.ac.in
```

Build the smallest service

1 Make and enter a folder

```
mkdir hello-service  
cd hello-service
```

You are now inside it. Check with pwd.

2 Create package.json

```
npm init -y
```

Prints the JSON it created. -y accepts all defaults.

3 Install Express

```
npm install express
```

"added 60-odd packages". A node_modules folder appears.

4 Create the file

```
In VS Code: New File,  
named server.js
```

Empty file. Type the nine lines from the next slide.

5 Start the server

```
node server.js
```

"listening on 3000". The prompt does NOT come back — correct.

6 Call it, from a SECOND terminal

```
curl.exe -i \  
http://localhost:3000/api/ping
```

HTTP/1.1 200 OK and {"status":"alive"}

7 Stop the server

```
Ctrl+C in the first  
terminal
```

The prompt returns. The port is released.

server.js — type this exactly

This is a web service.

```
const express = require("express");
const app = express();

app.use(express.json());

app.get("/api/ping", (req, res) => {
  res.json({ status: "alive" });
});

app.listen(3000, () => console.log("listening
on 3000"));
```

What you should see, in both terminals

```
# terminal 1
$ node server.js
listening on 3000

# terminal 2
$ curl.exe -i http://localhost:3000/api/ping
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{"status":"alive"}
```

require("express")

loads the library npm just installed

express()

creates your application object

app.use(express.json())

middleware: parses a JSON request body into req.body

app.get(path, handler)

when a GET arrives for this path, run this function

res.json(obj)

sends the object as JSON, and sets Content-Type for you

app.listen(3000, cb)

start listening on port 3000; run cb once ready

That is a web service

Nine lines. It listens on a port, matches a path, and returns JSON with a correct status and Content-Type.

Add a second route of your own before you leave.